

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO2 – Manipulation (BL3, BL4)	Assessment:	Assessment Tool 1 – Analytical Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO2 Statement:	<i>Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.</i>		
Student Name:	G. Koushik	Reg. No.:	192524238
Section / Batch:	1925 AIDJ	Date:	17/07/2026

Code of Conduct

I, G. Koushik (192524238) (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the Student:

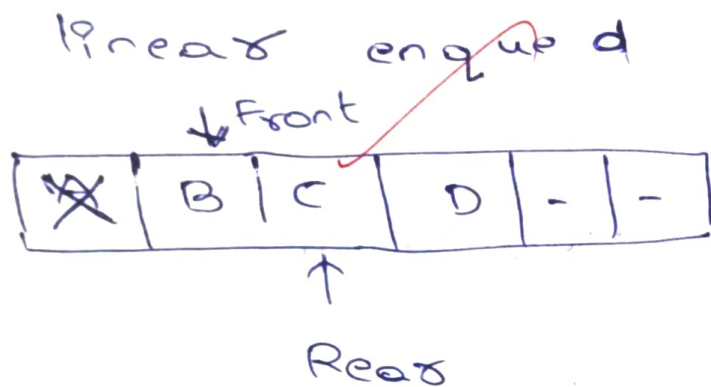
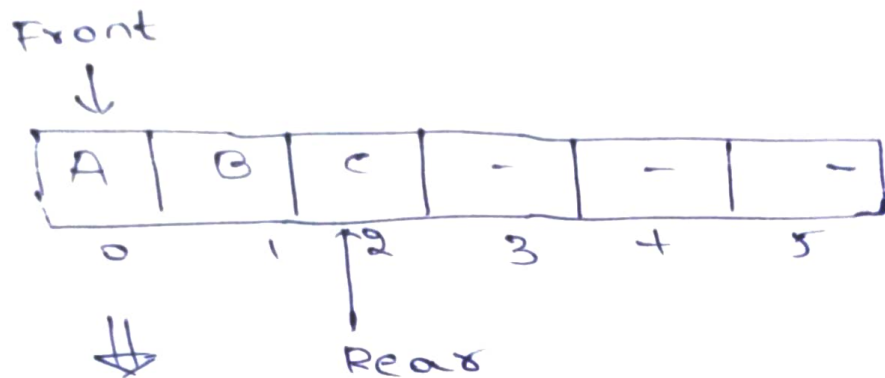
G. Koushik

Instructions

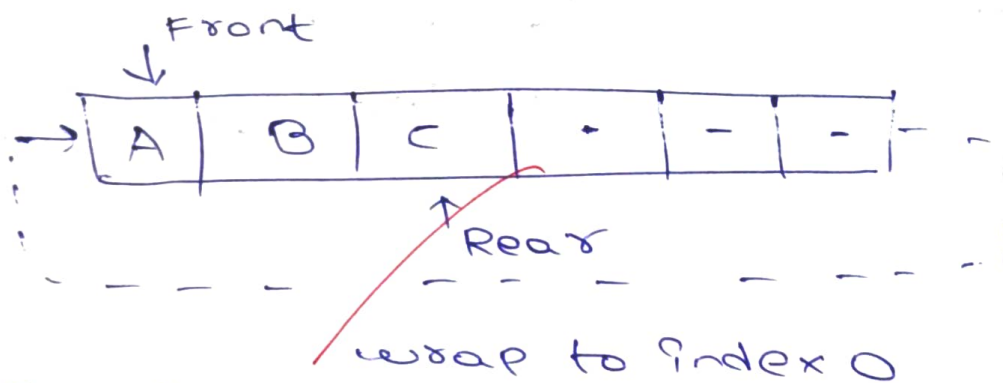
- This test contains 80 Analytical problem-solving questions. Total: 100 Marks.
- When a student answer using an Analytical problem-solving, in evaluation the answer should be with following five requirements: Understanding of problem, select appropriate data structure, Design the solution, analyze, Clarity of representation.
- Each student should write 2 questions. No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Linked Data Structure, Search Data Structure	List Operations, Binary Search	1	50
Stack Data Structure, Queue Data Structures	Stack Notations, Priority Queue	2	50
GRAND TOTAL:			100 Marks

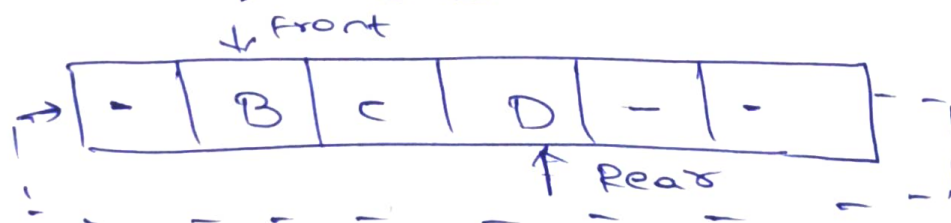
Assume the Queue has a fixed array size and 100 enqueue and dequeue operations are performed alternately



This is a linear Dequeue B



circular enqueue B



circular dequeue B

Feature	Linear Queue	Circular Queue
Enqueue operation	$O(1)$	$O(1)$
Dequeue operation	$O(1)$	$O(1)$
Space utilization	Poor	Better
Reuses deleted positions	NO	Yes
Performance for operations	May waste space	Efficient uses space
Array movement	May require shifting in some implementations	NO Shifting required
Overall efficiency	Lower space efficiency	Higher space efficiency

If every enqueue is followed by a dequeue, the no. of elements in the queue stays small.

Reason

In an array based linear queue, rear moves only in the forward direction

Initial:

[10] [20] [30] [40] [50]
F R

After deleting 10 and 20:

[] [] [30] [40] [50]
F R

The first two positions are empty, but rear is already at the last array position

$rear == Max - 1$

makes the queue report "Queue Full", even though there is unused space at the beginning.

This situation is called false overflow

b)

How does a circular queue solve it

A circular queue connects the last position back to the first position

When rear reaches the end, it wraps around to the beginning:

[30] [40] [50] [] []
↑ ↑
Front Rear

After enqueue

[30] [40] [50] [60] []

After another enqueue:

[30] [40] [50] [60] [70]

more generally, the next position is calculated using:

$$(\text{rear} + 1) \% \text{Max}$$

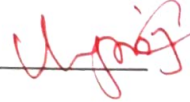
So, the empty positions created by dequeue operations can be reused

Linear Queue: can suffer from false overflow and waste array space

Circular Queue: Reuses empty positions

through wrap-around, giving better memory utilization and more efficient queue management.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20	Demonstrates complete and precise understanding; identifies all constraints and edge cases	Shows clear understanding with minor gaps	Partial understanding; misses some constraints	Misinterprets problem or lacks clarity
Data Structure Selection	20	Selects optimal data structure with strong justification and comparison	Selects appropriate structure with reasonable justification	Selects somewhat suitable structure with weak reasoning	Incorrect or no data structure selection
Algorithm Design	20	Designs efficient, logically sound, and well-structured algorithm	Algorithm is correct with minor inefficiencies	Algorithm is partially correct or lacks clarity	Algorithm is incorrect or missing
Accuracy of Solution	30	Error-free, wellstructured, optimized code/solution	Minor errors but overall functional	Multiple errors; partially working	Non-functional or missing solution
Clarity	10	Exceptionally clear, well-organized, uses diagrams effectively	Clear and organized with minor issues	Somewhat organized but lacks clarity	Poorly organized and unclear
Faculty In-charge		Course Coordinator		Program Director	
Signature: 		Signature: _____		Signature: _____	
Date: _____		Date: _____		Date: _____	